Chapter 3: Technical Background

NetDrop is a local-first web application designed for quick, temporary file, text, and link sharing over a Local Area Network (LAN). It also includes an online mode for connecting to a separately deployed HTTPS server. The technologies below describe what the system currently uses and what is planned for future development. Planned and optional tools are identified to distinguish them from implemented features.

3.1 Programming Languages

Python

Role in System: Python serves as the primary language for NetDrop's host server. It manages room creation, PIN verification, browser sessions, file operations, and background cleanup. Its asynchronous features support live connections and scheduled tasks. Local DNS and external notification delivery are planned additions rather than current Python functions in the system.

3.2 Backend Frameworks and Servers

FastAPI

Role in System: FastAPI is the web framework powering NetDrop's backend. It provides HTTP endpoints for creating and joining rooms, checking 6–12 digit PINs, sharing text and links, uploading files, and retrieving room members. File uploads use streamed binary request bodies. Pydantic models validate structured request data, and FastAPI also serves the browser interface.

Uvicorn

Role in System: Uvicorn is the ASGI (Asynchronous Server Gateway Interface) server that runs the FastAPI application. It accepts HTTP requests and maintains WebSocket connections. NetDrop currently uses one server process because rooms, browser sessions, membership, and content indexes are stored in that process's memory.

3.3 Frontend and Real-Time Communication

HTML5, CSS, and Plain JavaScript

Role in System: HTML, CSS, and JavaScript form the browser interface. They provide room forms, Local Network / Online tabs, file selection and uploads, shared notes and links, invitation links, and participant lists. JavaScript sends requests and handles live updates. Participants can use a phone, tablet, or computer browser without installing a separate NetDrop client.

WebSockets

Role in System: WebSockets provide real-time communication between browsers and the server. Changes to shared content or membership signal browsers to retrieve updates. Reconnection refreshes the room after an interruption. Active connections also indicate participant presence.

3.4 File Storage and Asset Retention

Local File System and In-Memory Storage

Role in System: NetDrop stores room metadata, sessions, and content indexes in memory, while uploaded files are saved in the host's data/uploads folder. The default upload limit is 100 MiB, and files, text, and links expire after 30 minutes. Cleanup normally runs every five seconds. Server restarts clear room indexes; expired orphaned uploads are removed by cleanup.

SQLite

Role in System: SQLite is an optional future database for persistent settings, notification subscriptions, activity logs, or restart-safe notification retries. It has not yet been integrated. The current system uses in-memory structures and temporary local files, so room and session information does not persist across server restarts.

3.5 Networking, DNS, and External Services

Python dnslib

Role in System: The dnslib library is an optional future tool for resolving a friendly local hostname to the host's LAN IP address. This requires suitable device or network DNS settings. NetDrop currently uses the host's address and port without a DNS server.

Greppoo Notification Hub API

Role in System: Greppoo Notification Hub is the planned external provider for new-file alerts. Its documentation, authentication, supported channels, and recipient setup must be verified before integration. The notification module is currently a placeholder. Planned alerts will exclude file contents and room PINs, and notification failures should not interrupt local sharing.

Python HTTPX

Role in System: HTTPX is currently a development dependency supporting FastAPI's test client. It is also planned for outbound notification API requests with bounded timeouts and retry handling. External alert delivery is not yet implemented or verified.

Cloudflare Tunnel (Optional Remote Access)

Role in System: Cloudflare Tunnel is an optional future method of making a local NetDrop host reachable through a public address. It is not integrated into the current system. The existing Online tab connects to a separately deployed HTTPS server. Each server retains its own rooms and uploads; switching servers does not synchronize their content.

3.6 Development, Packaging, and Deployment

Python Virtual Environment, pip, and python-dotenv

Role in System: A Python virtual environment and pip requirement files manage the project's dependencies. The python-dotenv library loads optional settings from a .env file. Configuration controls the host address, port, sharing mode, server URLs, upload size, expiry period, and cleanup interval.

unittest and Playwright

Role in System: Python's unittest framework and FastAPI's test client support checks for room access, sharing, expiry, and WebSocket behavior. Playwright scripts check browser workflows, including uploads, downloads, reconnection, and local/online interaction. These are development tools rather than software that participants must install.

Render and UptimeRobot

Role in System: The project documents Render as an online hosting target for the Python application. Online use requires a separately deployed HTTPS service with one server process. NetDrop's /up endpoint supports external availability checks, and the README provides UptimeRobot setup guidance. The current status of hosting and monitoring is not established by this document.

PyInstaller

Role in System: PyInstaller is an optional future packaging tool for bundling Python, backend dependencies, and static frontend files into a standalone host application. This would simplify launching NetDrop without manually setting up Python. Packages must be built and tested for each target operating system; no executable packaging workflow is currently implemented.